

FIPA final project 2025/26

1st Sjøen-Tollaksvik, Sebastian

FEUP

University of Porto

Porto, Portugal

up202503018@up.pt

I. ABSTRACT

This paper presents a simple yet effective tool design to assist experts in detecting Dry Eye Syndrome (DES) from a set of break-up time (BUT) test images. The algorithm in the exercise description is broken down into two stages: (i) extraction of the region of interest (the iris) using preprocessing techniques. As well as the Hough Circle Transform. (ii) detection of dark regions within the ROI through preprocessing, Otsu's thresholding and contour parameter tuning.

This approach relies solely on image processing techniques without any traditional machine learning or AI components, making it fast and intuitive. The model is designed to be flexible and able to adapt to new datasets. Results show that the method successfully removed noise by extracting the ROI (the iris). With the right parameter tuning, the model is also able to identify potential tear film break-up areas. It is important to remember that the model will have a lot of false positives. Therefore, this model is designed to serve as a supportive tool for experts rather than a classifier.

II. INTRODUCTION

This paper aims to develop an algorithm that experts can utilize as a tool to detect whether a patient has Dry Eye Syndrome (DES) by analyzing data from a break-up time test (BUT). DES can cause patients irritation, sensitivity to light, red eyes and even blurry vision. According to research conducted by the National Eye Institute in the US suggest nearly 16 million Americans suffer from this disease. Implementing an algorithm to assist with the process can not only save time but also improve results from check-ups.

The test inserts sodium fluorescein into the eye and measures the time taken from the appearance of dark regions until the eye blinks. This algorithm will help assist the experts as to when the dark areas appear. The algorithm is broken into two parts. (i) Extracting the region of interest, which in this case is the iris. (ii) identifying dark regions inside the region of interest that is within a pre-determined threshold.

III. TECHINICAL APPROACH

A. ROI extraction

Before we start to preprocess the images, it's important to understand the data we are working with. The first step is therefore to get a histogram of the pixel spread. The data here has an uneven spread with a lot of unspread images. This can be fixed using the `cv2.equalizeHist()` function. To get an even

better spread, we then use the Gaussian blur. We are still not happy with the spread, since there a lot of dark pixels. We fix this by defining the first ten percent of pixels to be valued as zero, so the model does not struggle to separate them.

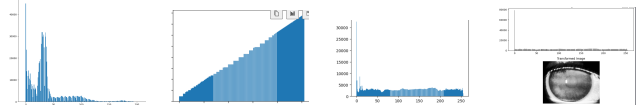


Fig. 1. (a) Original. (b) Histogram equalized. (c) Gaussian blur (d) Custom grayscale transformation.

1) Circle around the iris:

The next step is to identify the circle around the iris. This was probably the most difficult part to figure out during the whole exercise. After experimenting a lot with different edge detection techniques taught in class and heaps of research. I found the HoughCircles.

$$(x - a)^2 + (y - b)^2 = r^2 \quad (1)$$

The `cv2` imported function searches for (a, b, r) for each pixel. Afterward, there is a vote using canny edge detection to find the best number of pre-determined circles. In our case, we only want to find one circle, which is the iris. To further assist the function in finding the iris, we can also customize which interval of size we want the circle to be.

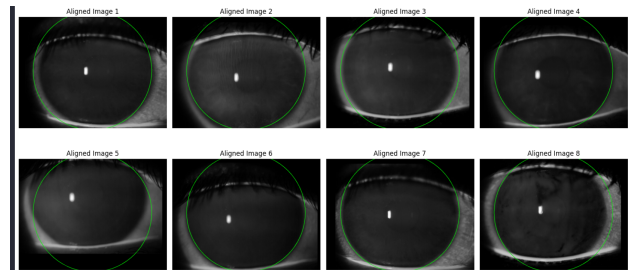


Fig. 2. Detected iris using Hough Circle Transform.

2) Further improvements and standardizing:

To further improve the results of iris detection, it is essential to minimize noise outside the iris. Especially the eyebrows, which contain a lot of dark pixels. There is also the issue of finding the same center for all circles to standardize the results. The first thing we do is shrink

the radius of the circles. It is not an issue if we do not catch the whole iris. It is a valid reward for us to remove noise from the eyebrows. The next step is to standardize the results. Therefore, we take the mean of the radius and the center point for all the images and apply it to all the images. This makes the function very flexible for different data and images. Even if the photos of the eyes are from different angles, the model would still be able to recognize the location of the iris. After doing all these preprocessing techniques, we now have access to the corresponding radius and center of the circle we will use to detect the iris from the images.

B. Dark region detection

The purpose of this step is to detect the dark regions which can possibly be dry eye syndrome, therefore we will also implement thresholds, which can be adjusted to detect whenever or not the dark regions are too small or large to be a dry eye syndrome. This will also make the functions more flexible and be a better tool for experts to use to identify dark regions within the eye.

1) Preprocessing image:

Similar to the first part of the exercise, we also need to preprocess the image in the second part of the exercise. The dark region iris detection function receives the original image, cropped to only fit the iris. This is because I was unsure whether the preprocessing was different from the first part. Since the image is mostly green, we start by extracting only the green channel, which improves the results by removing noise from the other channels. Afterward, the image gets normalized using the `cv2.equalizeHist()` function, just like in the iris detection subsection. To further improve the results there is applied a Gaussian blur function to remove noise from the image.

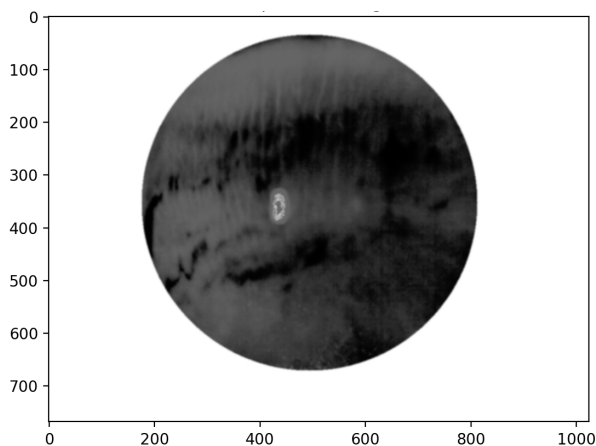


Fig. 3. Results of pre-processing ROI.

2) Identifying darkest regions within the eye:

Now that we have preprocessed the images and made an effort to remove the noise within the image,

the next logical step is to identify the dark regions within the iris. This is done in 4 different parts:

Otsu's thresholding:

automatically explores and finds the best intensity threshold to separate dark and bright regions. Using `cv2.THRESH_BINARY_INV`, the dark areas become white and the rest black.

Morphological operations:

even though we have found an optimal threshold, there will still be noise that bypasses both thresholding and preprocessing. Therefore, morphological opening and closing are applied (as taught in Lecture 8) to further reduce noise.

Bitwise AND:

keeps only the dark parts (now identified as white), removing everything else.

Canny edge detection:

Finally, we use Canny edge detection to find boundaries within the detected dark regions. This produces clear edges that make the last step possible.

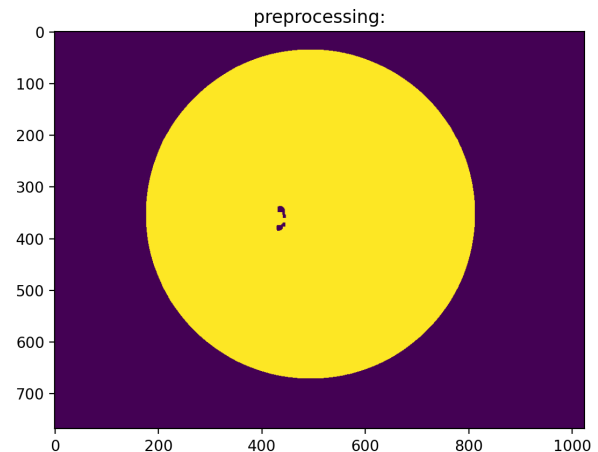


Fig. 4. Results of steps in subsection III.B.2.

3) Size threshold and implementation:

To better visualize the results, we use the powerful `cv2` contour function to better visualize our findings. The contours allow us to fill areas that are within a given size threshold. So we filter out every area that is too small and insignificant to be considered as DES. We also filter out the areas which are too large to be considered. This is usually noise, like edges around the iris and eyelashes, which overlap onto the iris and that we were not able to filter out in the earlier process.

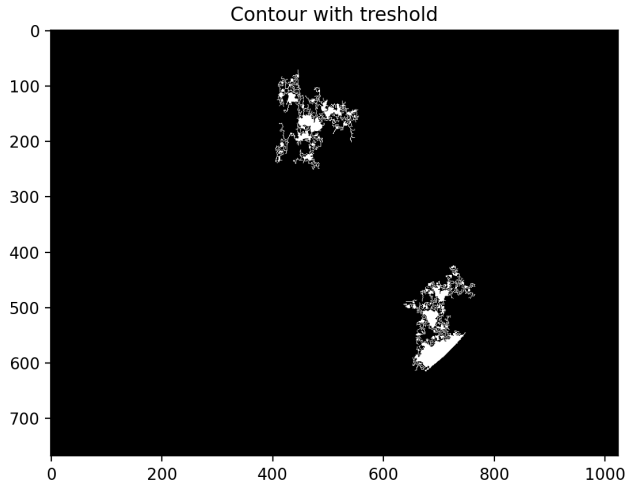


Fig. 5. Result of final DES detection.

IV. RESULTS

A. ROI extraction

Overall, the ROI extraction process was very successful in iris detection across most images. The combined preprocessing of histogram equalization, Gaussian blur, defining the darkest regions as the same value, and the Hough Circle Transform. Provided consistent and accurate circles that detect the iris. Using the mean radius and center further improved the consistency across samples. By taking the mean of all the data that is provided, the model is also flexible. Even if one expert takes images from a different angle than this dataset does, the model would still work. There is also a constant added within the preprocessing that shrinks the circle. For this dataset, this constant is set at $\times 0.69$ of the radius. This can also be modified for other datasets, so the model performs better.

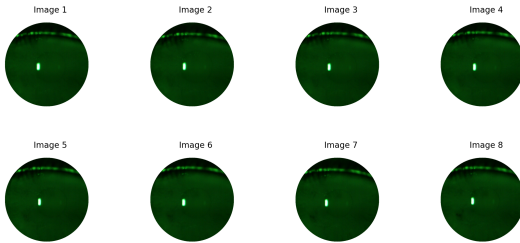


Fig. 6. Results of Task 1 ROI extraction.

B. Dark region detection

Overall, the dark region detection algorithm works fine. Through preprocessing steps such as green channel extraction, histogram equalization, and Gaussian blurring, the noise was significantly reduced, resulting in more reliable thresholding. The OTSU's method can find a reasonable threshold for

the individual images, making the model more robust and flexible on different images. Using the parameter tuning, the model detects dark areas within a given interval.

Although the algorithm works as intended, it is challenging to find optimal parameters to optimize the model. If you set the values too low, you will obtain many false results; however, if you set them too high, you may not detect the dark regions at all. When making this model, I had to decide how to visualize the groups of dark regions; there were mainly two options: K-means or contour. K-means is a powerful classifying machine learning method I have used before. The main problem with K-means is that you have to pre-determine a set amount of k clusters before you run the algorithm. The model will then create k amount of clusters and try to classify them into this set amount, even if there are more or fewer clusters. If you don't hit perfectly on the number of clusters there really are, the model will take the mean of the distances and create clusters where they don't really belong. Therefore, I decided to use contour even though k-means is a simple yet powerful classification model for images.

As previously mentioned, this model is not a classifier, since the algorithm is quite simple. The intended purpose is quite literally to identify dark regions within the eye; there will be a lot of false negatives if you want to include all the dark regions. Therefore, it's important to remember that this is not a classifier, but a tool to aid in the process of identifying dark regions within the eye for experts. Each image needs to be double-checked and classified by an expert.

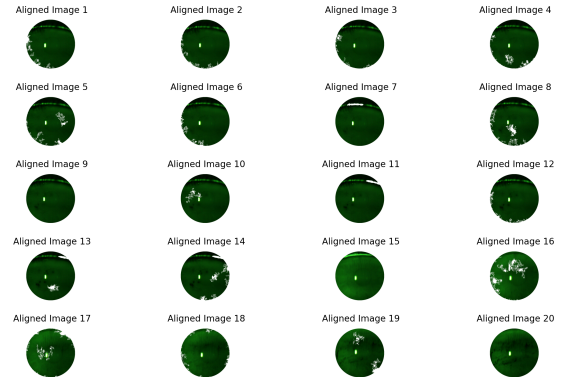


Fig. 7. Results of Task 2: Break-up analysis with sensitive parameter tuning.

V. CONCLUSION

This report explains a project to develop a tool to help detect dry eye syndrome (DRE), by detecting dark regions within the eye. The project is split into two parts: extraction of the region of interest (ROI) and identification of dark regions within a given

interval inside the ROI. Using methods taught in class as well as some other strong cv2 functions, the models performed quite well for their intended purposes. It is important to remember that this is a supportive tool for experts to use and parameter-tune to find potential dark spots within the iris.

REFERENCES

- [1] National Eye Institution, "At a glance: Dry Eye," August 6, 2025.
<https://www.nei.nih.gov/learn-about-eye-health/eye-conditions-and-diseases/dry-eye>
- [2] Doxygen, "Feature detection, image Processing" October 29 2025
https://docs.opencv.org/4.x/dd/d1a/group__imgproc__feature.html#ga47849c3be0d0406ad3ca45db65a25d2d
- [3] Numpy developers 2008-2024 "numpy.bitwise and"
https://numpy.org/doc/2.2/reference/generated/numpy.bitwise_and.html